

Aufgabenblatt Color Selector

16. Mai 2026

1 Aktueller Code

1.1 *my-first-app/src/main/java/de/lyzeum/web/MyFirstView.java*

```
package de.lyzeum.web;

import com.vaadin.flow.component.html.H1;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.slider.Slider;
import com.vaadin.flow.router.PageTitle;
import com.vaadin.flow.router.Route;

@Route("/color-selector")
@PageTitle("Color Selector")
public class ColorSelectorView extends VerticalLayout {
    private final Slider redSlider;

    public ColorSelectorView() {
        H1 title = new H1("Color Selector");
        redSlider = new Slider("Red", 0, 255);
        add(title, redSlider);
    }
}
```

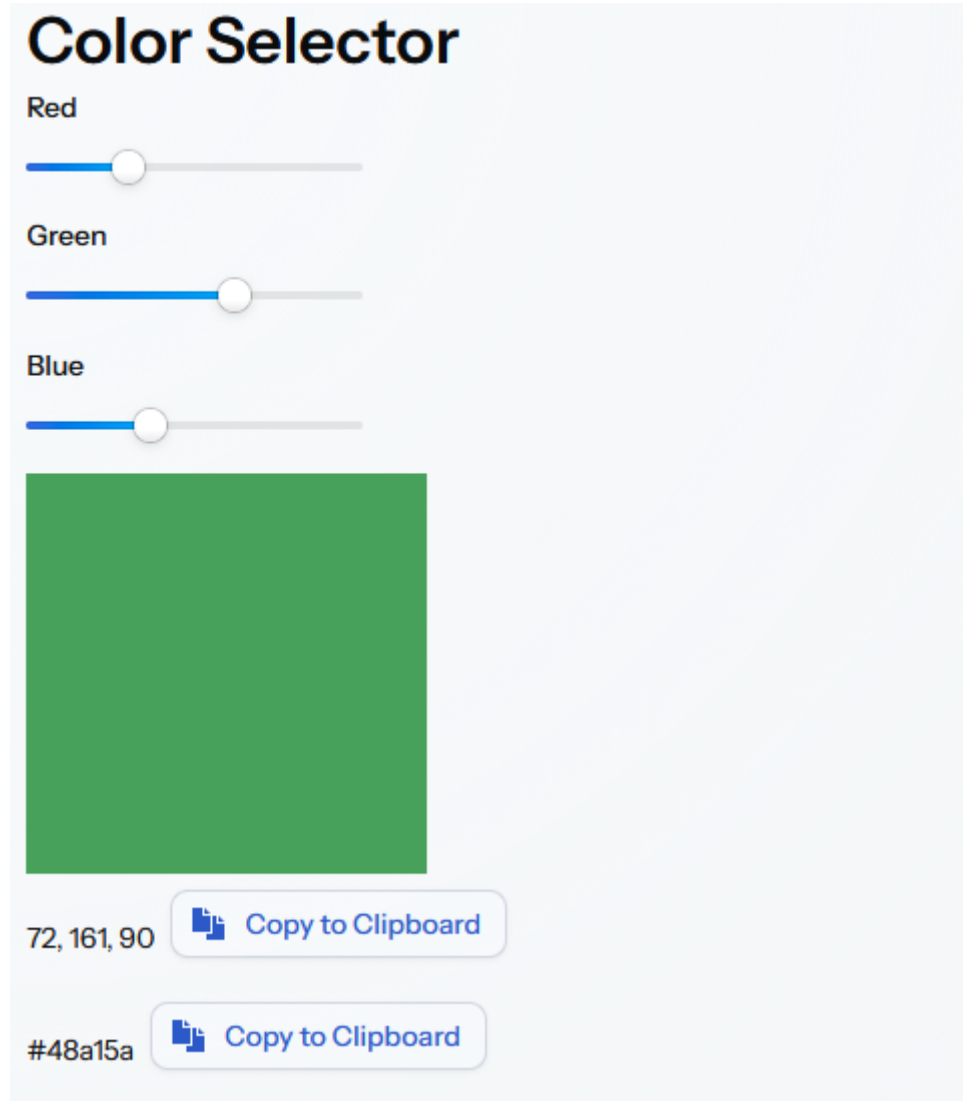
Die Annotation `@Route` beschreibt den Pfad der neuen Webseite, in diesem Fall setzt sich dieser aus der Basis-URL `http://localhost:8080` und dem Pfad `/color-selector` zum endgültigen Wert `http://localhost:8080/color-selector` zusammen. Auf der Webseite ist bisher eine Überschrift und ein Slider zu sehen. Der Slider ist für die Farbe Rot gedacht und besitzt einen erlaubten Wertebereich zwischen 0 bis 255. Der Aufruf der `add()`-Funktion fügt alle erzeugten Elemente der Webseite hinzu.

1.1.1 *my-first-app/src/main/resources/vaadin-featureflags.properties*

Die folgende Zeile ist notwendig um die Komponente Slider in Vaadin zu aktivieren. Nach aktuellen Stand ist dieses Feature noch in einer experimentellen Phase und muss deshalb per Konfiguration dem Projekt hinzugefügt werden.

```
com.vaadin.experimental.sliderComponent=true
```

2 Zielzustand



Die Webseite besteht aus drei auswählbaren Slidern, welche den Rot-, Grün- und Blauanteil des darunter liegenden Elementes steuern. Zusätzlich werden die eingestellten Parameter als RGB- und als Hexwert dargestellt, welche mit zwei nebenstehenden Buttons in die Zwischenablage kopiert werden können.

3 Aufgaben

3.1 Weitere Slider

Füge die zwei neuen Slider `blueSlider` und `greenSlider` hinzu, welche unterhalb des roten Sliders platziert werden. Überlege, ob diese Slider als Klassenattribut oder als lokales Attribut innerhalb des Konstruktors deklariert werden müssen. Setze dazu passende Beschriftungen und den Wertebereich der Grün- und Blauanteile

3.2 Farbensvorschau

Füge als nächstes ein Div-Element mit dem Variablennamen `colorContainer` hinzu, welches die Größe 200 x 200 in Pixeln besitzen soll. Ein Div-Element enthält normalerweise weitere Elemente (z.B. Texte und Buttons), aber in diesem Fall soll es nur zum Darstellen von Farbwerten verwendet werden. Überlege, ob dieses Element als Klassenattribut oder als lokales Attribut definiert werden muss. Für die Größenangabe könnte folgendes CSS-Styling helfen:

```
.color-container {  
    width: 200px;  
    height: 200px;  
}
```

3.3 Farbanpassungen nach Slider-Update

Jeder der Slider soll bei einer Änderung eine Farbaktualisierung vom Div-Element auslösen. Dafür soll folgende Methode angepasst werden:

```
public void onSliderUpdate() {  
    // TODO echte Farbwerte aus Slider extrahieren und String zusammenbauen  
    // redSlider.getValue().intValue()  
    // TODO Wert durch echten Farbwert ersetzen  
    colorContainer.getStyle().setBackgroundColor("rgb(255, 0, 0)");  
}
```

Für den Aufruf der neuen Funktion kann an einer passenden Stelle im Konstruktor folgender Code hinzugefügt werden:

```
redSlider.addValueChangeListener(event -> onSliderUpdate());  
redSlider.setValueChangeMode(ValueChangeMode.EAGER);
```

Hinweis: Die erste Zeile löst nach jeder Wertänderung des Sliders ein Event aus, welches in diesem Fall die Funktion `onSliderUpdate()` aufruft. Die zweite Zeile spezifiziert, dass sofort nach dem Bewegen des Sliders eine Wertänderung erkannt wird. Im Normalfall würde die Wertänderung erst nach Loslassen des Mausbuttons ein Event auslösen.

3.4 Darstellung des Farbwertes als Label

Nachdem nun die Farbe innerhalb des Div-Elementes nach dem Bewegen der Slider sich ändern sollte, wäre eine weitere sinnvolle Information die Anzeige des aktuellen RGB-Wertes. Erstelle ein neues Paragraph-Element mit dem Variablennamen `rgbParagraph` und füge dieses Element der Webseite hinzu. Dieses Element soll anhand der Slider-Werte den genauen RGB-Farbwert in Textform (z.B. 255, 0, 0 für die Farbe Rot) anzeigen. Aktualisiere dafür die Funktion `onSliderUpdate()` um den dort erstellten RGB-Farbwert zusammenzubauen und als Wert des Paragraphen zu setzen.

Hinweis: Der Paragraph soll bereits beim Aufruf der Webseite als Text den Farbwert des Div-Elementes anzeigen. Wie könnte das geschickt gelöst werden?

3.5 Copy-To-Clipboard Funktion

Diese Funktion benötigt neben Java-Code-Änderungen auch ein etwas JavaScript-Code. Erstelle dazu zuerst unter dem Pfad `src/main/frontend/copytoclipboard.js` die neue Datei `copytoclipboard.js`. Anschließend soll folgender Code eingefügt werden:

```
window.copyToClipboard = (str) => {
  const textarea = document.createElement("textarea");
  textarea.value = str;
  textarea.style.position = "absolute";
  textarea.style.opacity = "0";
  document.body.appendChild(textarea);
  textarea.select();
  document.execCommand("copy");
  document.body.removeChild(textarea);
};
```

Füge vor Beginn der Klassendefinition folgende neue Annotation hinzu um den JS-Code verfügbar zu machen:

```
@JsModule("./copytoclipboard.js")
public class ColorSelectorView extends VerticalLayout {
    // ...
}
```

Erstelle danach im Konstruktor einen neuen Button `copyRgbButton` (oder wähle einen besseren Namen, falls dir dieser Wert nicht gefällt). Wähle als Text den Wert `Copy to Clipboard`.

Hinweis: Der Button soll neben dem Paragraph dargestellt werden.

Füge anschließend einen neuen ClickListener hinzu, welcher den RGB-Wert nimmt und damit auf folgende Weise den JavaScript-Code aufruft:

```
UI.getCurrent().getPage().executeJs("window.copyToClipboard($0)", rgbParagraph.getText())
```

Hinweis: Verwende nach Möglichkeit eine innere Funktion. Überlege, ob der Button als Klassenattribut oder lokales Attribut definiert werden muss.

3.6 Bonusaufgaben

3.6.1 Icons

Das Enum `VaadinIcon` bietet zahlreiche Icons an, welche auf einer Vaadin-Webseite dargestellt werden können. Auf dieser [Webseite](#) können die verfügbaren Icons betrachtet werden. Ersetze nun mithilfe des gezeigten Beispiels deinen Button um auch ein Icon darzustellen.

```
Button exampleButton = new Button("Copy to Clipboard", (VaadinIcon.COPY.create()));
```

3.6.2 Hexwert-Darstellung

Füge einen weiteren Paragraphen `hexParagraph` hinzu, welcher den RGB-Wert als Hexwert (z.B. `#FFAB12`) darstellt. Dieser soll wie beim RGB-Wert beim Aufruf von `onSliderUpdate()` den aktuellen Farbwert in Textform darstellen. Ferner soll auch hierfür ein Copy-To-Clipboard-Button erstellt werden.